Exchange Online DLP — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Exchange Online DLP in Microsoft Purview 4
- The problem . . . 4
- The solution . . . 4
- How it works — the model . . . 4
- What you get . . . 4
- Scope and boundaries . . . 5
- The bottom line . . . 5

Solution Overview — Exchange Online DLP in Microsoft Purview 5
- What this solution delivers . . . 5
- What Exchange Online DLP is (and is not) . . . 6
- The scenarios this solution controls . . . 6
- Simulate first — this is a high-impact control . . . 7
- The standard deployment model . . . 7
- What's in the box . . . 7

Requirements — Exchange Online DLP in Microsoft Purview 8
- R1. DLP-capable licensing for Exchange, assigned to in-scope senders . . . 8
- R2. Least-privilege roles for DLP authoring (and mail-flow awareness) . . . 8
- R3. In-scope sensitive content — sensitive information types and/or sensitivity labels . . . 9
- R4. Recipient-scope decision (external vs internal, trusted-partner domains) . . . 9
- R5. Per-scenario action decision (notify / warn / encrypt / block) — agreed with stakeholders . . . 9
- R6. Policy-tip and notification plan . . . 10
- R7. Transport-rule (mail-flow) interaction reviewed . . . 10
- R8. A named pilot sender scope and change control (simulation/test first) . . . 10

Global Requirements — Microsoft Purview Solutions 11
- 1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11
- 2. Purview roles (least-privilege) and portal access . . . 11
- 3. Security & Compliance PowerShell + Graph toolchain . . . 12
- 4. Data sources & scope in scope . . . 12
- 5. Compliance, records, privacy & legal stakeholders . . . 12
- 6. A pilot scope and change control . . . 12

Implementation Guide — Exchange Online DLP in Microsoft Purview 13
- Phase 0 — Assess (confirm you are ready) . . . 14
- Phase 1 — Design (name every object, agree every action) . . . 14
- Phase 2 — Configure the policy and rules in SIMULATION . . . 15
- Phase 3 — Pilot & validate in simulation . . . 16

Phase 4 — Enforce, then expand . . . 16
Rollback summary (fast reference) . . . 17
Testing & Validation — Exchange Online DLP 17
A. Configuration gates (automated — `DLPEXO-Validate.ps1`, read-only) . . . 18
B. Behavioural tests — per scenario (manual, from a pilot mailbox) . . . 18
C. Negative / false-positive tests (both passes) . . . 19
D. Evidence to capture (for the defensibility pack) . . . 20
Troubleshooting — Exchange Online DLP 20
1. A test message doesn't match at all . . . 20
2. Policy tips don't appear in Outlook . . . 21
3. The rule fires on the wrong things (false positives) . . . 21
4. Simulation looks fine but enforcement doesn't block/encrypt . . . 22
5. The encrypt action doesn't apply (or errors) . . . 22
6. `New-DlpComplianceRule` fails on a condition/action parameter . . . 23
7. DLP and an Exchange transport (mail-flow) rule conflict . . . 23
8. Multiple DLP policies overlap / precedence confusion . . . 23
9. Activity report is empty or export fails . . . 23
10. "It blocked an exec / broke a business flow" — emergency rollback . . . 24
When to escalate to the determination owners (not a technical fix) . . . 24
Best Practices — Exchange Online DLP 24
Deploy the way the house model says — simulate, then enforce . . . 24
Key the rules on *sensitive* content and the right recipient scope . . . 25
Maintain the trusted-partner exception deliberately . . . 25
Use policy tips as user education, not just enforcement . . . 25
Reconcile with Exchange mail-flow (transport) rules . . . 26
Operate it, don't set-and-forget . . . 26
Keep the roles and determinations where they belong . . . 26
Security & handling of the tooling . . . 26
Deliverables Summary — Exchange Online DLP in Microsoft Purview 27
Documents . . . 27
PowerShell tools (5) . . . 28
How it maps to the deployment model . . . 29
Definition of done . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary — Exchange Online DLP in Microsoft Purview

The problem

Email is the top channel through which sensitive data leaves an organization — a list of card numbers or national IDs attached to a message, a *Confidential* contract forwarded to an outside party, a spreadsheet sent to a personal address. Left unaddressed, outbound email carrying regulated or classified data is both a **compliance gap** and a direct **data-risk exposure**, and by default there may be **nothing inspecting it** before it leaves.

The solution

This solution delivers **Microsoft Purview Data Loss Prevention (DLP) for Exchange Online** end to end. It inventories what already exists, then authors one Exchange DLP policy with rules covering the core email scenarios — **sensitive information to external recipients** (warned at low volume, blocked at high volume), **sensitivity-labeled mail to external recipients** (encrypted), and **sensitive information to internal recipients** (audited) — each keyed to your sensitive content (sensitive information types and/or sensitivity labels) and to the right recipient scope, with a **trusted-partner exception** so sanctioned business flows are not disrupted, and **policy tips** that guide the sender at the moment of *Send*. Crucially, every rule is **piloted in simulation/test mode first** — it records what it *would* do without blocking or encrypting anyone — validated on a small sender ring, and only then moved to **enforce**.

How it works — the model

The delivery follows a disciplined, repeatable model with an explicit pass/fail gate at each step:

Assess → **Design** → **Configure** → **Pilot (simulation)** → **Validate** → **Expand (enforce).**

Because a DLP block stops a real message and an encrypt action changes how it is delivered, nothing is enforced until it has been proven, in simulation, to match the right mail and act only on the right content — with a one-command rollback available at all times.

What you get

- A complete document set: Overview, Requirements, an ordered Implementation guide, Testing (pass/fail

gates), Troubleshooting, Best Practices, and this summary.

- **Five PowerShell tools** (Security & Compliance PowerShell) — discovery, configuration (simulation, `-WhatIf` first, writes gated by `-Commit`), validation, reporting, and scope/health — all **read-only or preview by default, with no credentials in code.**

- A **defensibility evidence pack**: proof of what was configured, that each scenario was validated in simulation before enforcement, and that enforcement behaves as designed.

Scope and boundaries

• DLP is **licensing-dependent** (advanced conditions/actions and message encryption may require Microsoft 365 E5 / E5 Compliance and enabled Purview Message Encryption) and the Purview portal **evolves** — capability availability and exact paths/parameters are **verified against your licensing/tenant**, never assumed.

• Exchange DLP **coexists with Exchange mail-flow (transport) rules**; overlaps are reconciled so the two controls do not conflict.

• **Regulatory, records, and legal determinations** — which data is sensitive, which scenarios must be blocked versus audited, whether a recipient domain is trusted, and whether a message is a violation — remain with your **compliance, security, records, privacy, and legal** functions. This solution provides the **technical method and the evidence**, not the determination.

The bottom line

Unmonitored outbound email becomes a **monitored, then enforced** control set in Microsoft Purview — **configured, piloted in simulation, validated, and defensible.**

Solution Overview — Exchange Online DLP in Microsoft Purview

What this solution delivers

Email is the single most common channel through which sensitive data leaves an organization. A credit-card list pasted into a message, a spreadsheet of national IDs attached and sent to a personal address, a *Confidential*-labeled contract forwarded to an outside party — every one of those is an outbound email that today may leave with **nothing watching it**. **Microsoft Purview Data Loss Prevention (DLP) for Exchange Online** closes that gap: it inspects messages in the transport pipeline against your **sensitive content** and can **notify**, **warn (with override)**, **encrypt**, or **block** the message before it leaves — and it can show the sender a **policy tip** explaining why, at the moment they hit *Send*.

This solution walks a competent-but-novice administrator through a **complete, repeatable Exchange Online DLP deployment**:

1. **Discover what already exists** — inventory any DLP policies already scoped to Exchange, how they interact with **Exchange mail-flow (transport) rules**, which sensitive information types (SITs) and sensitivity labels are available to key on, and the current policy **modes**.

1. **Design the policy** — decide, with your compliance/security stakeholders, *which* sensitive content matters (SITs, sensitivity labels), *which* recipient scope is risky (external vs internal, trusted-partner domains), and *which* action each scenario takes (**notify / warn / encrypt / block**).

1. **Author one Exchange Online DLP policy** with a small set of rules covering the core scenarios — sensitive content to **external** recipients, sensitivity-labeled mail to external recipients, internal visibility — **piloted in simulation/test mode first** so nothing is blocked until you have validated it, and with **policy tips** configured for user guidance.

1. **Validate, then enforce and expand** — prove in simulation that each rule *sees* the right mail and *would* act on the right content, then move the rules from simulation to **enforce**, one scenario at a time, with an explicit pass/fail gate and a documented rollback at every step.

The result turns **unmonitored outbound email** into a **monitored, then enforced** control set on Exchange Online — configured, piloted, validated, and defensible in Microsoft Purview.

What Exchange Online DLP is (and is not)

- **It is** the application of Purview Data Loss Prevention to the **Exchange** location — inspecting message content, attachments, and properties in the transport pipeline and taking a rule action (notify, policy tip, allow-with-override, encrypt, block, incident report) when a message matches your conditions.
- **It is not** the same thing as an **Exchange mail-flow (transport) rule**. Both act on mail in transport and *both can coexist*, but Purview DLP is authored in the **Purview compliance** surface (policies + rules over the whole M365 estate), evaluates alongside transport rules, and is the strategic home for content-based data-loss control. This solution uses **Purview DLP**; it flags where transport-rule interaction matters (see `06-troubleshooting.md`).
- **It is not** a substitute for your SharePoint/OneDrive/Teams/Endpoint DLP — Exchange is **one location**. A single DLP policy *can* span multiple locations; this solution focuses on the** Exchange location.
- **It is licensing-dependent.** Baseline DLP for Exchange is broadly available, but **advanced capabilities** (some conditions/classifiers, simulation reporting depth, certain encryption actions) can require **Microsoft 365 E5** or the **E5 Compliance** add-on, and the portal is **evolving** — **verify capability availability and exact portal paths/cmdlet parameters against your licensing/tenant** rather than assuming (see `global-requirements-purview.md` §1).

The scenarios this solution controls

Each is an email condition the DLP policy can detect and, in a rule, **Notify (policy tip)**, **Warn with override**, **Encrypt**, or **Block**:

Scenario	What the rule keys on
Sensitive info → external (high volume)	A message containing a sensitive information type (e.g. credit card, national ID) above a chosen instance count, sent to recipients **outside** the organization.
Sensitive info → external (low volume)	The same, at a lower instance count — usually **warn with policy tip / allow override** rather than hard block.
Labeled mail → external	A message (or attachment) carrying a **sensitivity label** such as *Confidential*, sent externally — often **encrypt** or **block**.
Sensitive info → internal	Sensitive content sent to **internal** recipients — usually **audit / notify** for visibility, not block.

Scenario	What the rule keys on
Trusted-partner exception	Recipient domains you explicitly trust are treated as an **exception** so sanctioned business flows are not blocked.

> ***Policy tips.*** *When a rule is configured to notify the sender, Exchange can display a* ***policy tip*** *in Outlook (desktop/web) at compose time — explaining the issue and, where configured, offering* ***override with business justification*** *or* ***report false positive****. Policy tips are a user-education control, not just an enforcement control.* ***Verify tip behavior per Outlook client in your tenant.***

Simulate first — this is a high-impact control

A DLP **block** action stops a real message from being delivered. A mis-scoped rule can block legitimate business email mid-flow. Therefore **every** policy in this solution is deployed in **simulation/test mode first**. Exchange Online DLP policies support four modes — `Enable` (enforce), `Disable` (off), `TestWithNotifications` (simulate *and* show policy tips), and `TestWithoutNotifications` (simulate silently). In a Test mode the policy **evaluates and records what it *would* do without blocking or encrypting anyone** — validated on a pilot sender scope, and only then moved to `Enable`. **Verify the exact mode names/behavior in your tenant** (the portal wording evolves).

> ***Determinations are not yours to make.*** **Which* data is sensitive, *which* scenarios must be* ***blocked*** *versus merely* ***audited****, *whether* a given message is a policy violation, and *whether* a recipient domain is "trusted" are* ***compliance / security / privacy / legal*** *determinations — this solution delivers the technical configuration and the evidence, not the determination (see* `global-requirements-purview.md` *§5).*

The standard deployment model

Every step follows the same repeatable model, with an explicit **pass/fail gate** between phases:

Assess → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand (enforce).**

- **Assess** — confirm licensing, least-privilege roles, in-scope sensitive content (SITs/labels), the recipient-scope decisions, the pilot sender scope, and compliance/security stakeholder sign-off.
- **Design** — decide which scenarios **notify** vs **warn** vs **encrypt** vs **block**, name every policy/rule/object, define the trusted-partner exception, and set the pilot scope.
- **Configure** — build the policy and its rules **in simulation/test mode**, with policy tips.
- **Pilot** — run in simulation on the pilot senders; confirm each scenario is seen and *would* act.
- **Validate** — run the pass/fail gates in `05-testing.md`.
- **Expand** — move rules from simulation to enforce and widen scope **only** after the gate passes; roll back per `03-implementation.md` if a gate fails.

What's in the box

- This overview, a **Requirements** guide, an ordered **Implementation** guide, **Testing**

(pass/fail gates), **Troubleshooting**, **Best Practices**, and a **Deliverables Summary** + **Executive Summary**.

- **Five PowerShell tools** (Security & Compliance PowerShell via `Connect-IPPSSession`): a

Discovery inventory of existing Exchange-scoped DLP config, transport-rule interaction, SITs and labels, and current modes; a **Configure/Remediation** helper that builds the policy + rules in **simulation (`-WhatIf` first, writes gated by `-Commit`)**; a **Validation** checker; a **Reporting** rollup of DLP rule matches / policy-tip / block events; and a **scope/health** helper — **read-only or `-WhatIf` by default; no credentials in code.**

> ***Sources.*** *Microsoft Learn — *Learn about data loss prevention*, *Create and deploy data loss prevention policies* (conditions, actions, simulation mode), *Data loss prevention policy tips reference*, *DLP policy conditions and actions for Exchange*, and the Security & Compliance PowerShell reference for `*-DlpCompliancePolicy` / `*-DlpComplianceRule` / `Get-DlpSensitiveInformationType`. DLP capability is **licensing-dependent** and the Purview portal is **evolving — verify capability availability, portal paths, and cmdlet parameters against your licensing/tenant** rather than assuming.*

Requirements — Exchange Online DLP in Microsoft Purview

Confirm every prerequisite below **before** you configure anything. Category-level prerequisites (tenant, compliance licensing model, least-privilege roles, Security & Compliance PowerShell / Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, pilot scope + change control) are in `global-requirements-purview.md` — reference them; they are **not** repeated here. The items below are the ones **specific to Exchange Online DLP**: the **content to key on**, the **recipient-scope decisions**, **policy tips**, **transport-rule interaction**, and the **pilot sender scope**.

Each requirement is written as **Description · Why · Verify · Implement if missing.**

R1. DLP-capable licensing for Exchange, assigned to in-scope senders

Description: Licensing that includes **Data Loss Prevention for Exchange Online**. Baseline DLP is broadly available, but **advanced** conditions/classifiers, deeper simulation reporting, and some **encryption** actions can require **Microsoft 365 E5** or the **E5 Compliance** add-on. Message encryption actions additionally depend on **Microsoft Purview Message Encryption (OME) / Azure Rights Management** being enabled. **Why:** DLP is plan-gated at the advanced tier, and the encrypt action will not be available unless message encryption is enabled in the tenant. **Verify:** Microsoft 365 admin center → **Billing** → **Licenses** confirms the SKU/add-on; in the Purview portal, **Data Loss Prevention** exposes the **Exchange email** location. Confirm message encryption is enabled (`Get-IRMConfiguration` / `Get-OMEConfiguration`) before relying on an encrypt action. Cross-check advanced features against the *Microsoft Purview service description* — **verify against your licensing/tenant. Implement if missing:** Acquire/assign the required SKU/add-on and enable Purview Message Encryption before authoring encrypt rules.

R2. Least-privilege roles for DLP authoring (and mail-flow awareness)

Description: The least-privilege roles, mapped to **named** people:

• **Compliance Administrator** or **Compliance Data Administrator** (or the **DLP Compliance Management** role group) to author the DLP policy and rules.

• Read access to **Exchange mail-flow (transport) rules** (e.g. an Exchange **View-Only Organization Management** role) so you can see how existing transport rules interact with DLP.

• **Global Administrator** only where genuinely required (e.g. first-time service enablement or enabling message encryption). **Why:** Purview uses fine-grained role groups; DLP authoring and Exchange mail-flow administration are distinct duties and should not be over-granted to one account. **Verify:** Purview portal → **Roles & scopes / Permissions** confirms the DLP role membership; `Connect-IPPSSession` succeeds and `Get-DlpCompliancePolicy` returns. `DLPEXO-Discovery.ps1` reports the DLP and transport-rule signals it can read. **Implement if missing:** Assign the least-privilege role for each duty under change control.

R3. In-scope sensitive content — sensitive information types and/or sensitivity labels

Description: The **sensitive content** the rules will match — **sensitive information types** (built-in such as *Credit Card Number*, *U.S. Social Security Number (SSN)*, or custom SITs / EDM / trainable classifiers) **and/or sensitivity labels** (e.g. *Confidential*) already published and in use — that define *what* triggers each rule, plus the intended **instance count / confidence** thresholds. **Why:** A DLP rule acts on **content that matches a condition**; without SITs or labels to match, the rule has nothing to key on (or would apply to *all* mail, which is rarely intended). **Verify:** Purview → **Data classification** → **Sensitive info types** lists the SITs; **Information Protection** → **Labels** shows published labels. `DLPEXO-Discovery.ps1 -IncludeSensitiveTypes` inventories both. Confirm the *count/confidence* thresholds with your data owners. **Implement if missing:** Select/adopt the needed SITs (and confidence/count thresholds) and, where you will key on labels, confirm the sensitivity labels are published. **Regulatory applicability of a given SIT is a compliance determination — not a technical default.**

R4. Recipient-scope decision (external vs internal, trusted-partner domains)

Description: A documented decision on **which recipient scope is risky** — typically **external** (outside the organization) is the primary concern, with **internal** handled for visibility — and an explicit list of **trusted-partner domains** to treat as an **exception** so sanctioned business flows (e.g. an outsourced payroll processor) are not blocked. **Why:** DLP rules key off recipient scope; blocking *all* external mail containing a SIT is usually too broad, and a missing partner exception will block legitimate business the moment you enforce. **Verify:** Confirm the external/internal scope and the trusted-partner domain list **in writing** with compliance/security. **Whether a domain is "trusted" is a business/compliance determination**, captured here, not chosen by the administrator. **Implement if missing:** Capture the recipient-scope decision and the partner-domain exception list before authoring the rules that reference them.

R5. Per-scenario action decision (notify / warn / encrypt / block) — agreed with stakeholders

Description: A documented decision, **per scenario** (sensitive→external high volume, sensitive→external low volume, labeled→external, sensitive→internal), of the intended action — **Notify (policy tip)**, **Warn**

with override / business justification, **Encrypt**, or **Block** — agreed with **compliance / security** (and privacy/legal where the action affects users or business partners). **Why:** The action is a **policy decision, not a technical default**; encrypting or blocking a message has real operational and business impact and must reflect enterprise policy, not the administrator's preference. **Verify:** Confirm the per-scenario action matrix **in writing** before you move any rule from simulation to enforce. **Implement if missing:** Capture the decision with the relevant owners; until then, deploy **notify / simulation only.** (See `global-requirements-purview.md` §5.)

R6. Policy-tip and notification plan

Description: The **policy-tip** and **notification** design: the custom **policy-tip text** shown to senders in Outlook, whether senders may **override** (with/without business justification) or **report a false positive**, whether the **sender** and/or a **compliance mailbox / admin** receive an email notification, and whether an **incident report** is generated. **Why:** Policy tips and notifications are the user-education and evidence layer; a block with no explanation generates tickets and workarounds, and an unrecorded match yields no evidence. **Verify:** Confirm the tip text, override options, and notification/incident-report recipients with stakeholders. **Verify policy-tip rendering per Outlook client (desktop/web/mobile) in your tenant** — tip support varies by client. **Implement if missing:** Draft the tip text and notification/override plan before authoring the rules; a compliance mailbox for notifications/incident reports should exist first.

R7. Transport-rule (mail-flow) interaction reviewed

Description: A review of existing **Exchange mail-flow (transport) rules** — especially any that already **block, redirect, encrypt, or add headers** based on content or recipient — so the new DLP policy does not **duplicate, conflict with, or be pre-empted by** them (e.g. a transport rule that already deletes a message before DLP evaluates it, or two controls both stamping encryption). **Why:** DLP and transport rules **coexist** and both act in transport; overlap causes confusing double-actions or silent pre-emption that looks like "DLP isn't working." **Verify:** Review transport rules (`Get-TransportRule`, or Exchange admin center → **Mail flow** → **Rules**). `DLPEXO-Discovery.ps1` flags transport rules that reference content/encryption/block actions for you to review. **Determining which control owns a given behavior is a design decision — capture it. Implement if missing:** Reconcile overlaps **before** enforcing DLP — decide which control owns each behavior and remove/adjust the duplicate.

R8. A named pilot sender scope and change control (simulation/test first)

Description: A **named pilot sender scope** (a mail-enabled security group of pilot senders, e.g. `SG-DLPEXO-Pilot`, used at the **rule level** via a sender condition) and agreement to run the policy in **simulation/test mode first** (`TestWithNotifications` / `TestWithoutNotifications`), under a **change-control** process, before any enforcing, tenant-wide rollout. **Why:** DLP block/encrypt actions are high-impact; simulation scoped to a small sender ring proves the scenarios are seen and the rules *would* act **without** disrupting real mail flow, and lets you tune before enforcing. **Verify:** Confirm the pilot group exists, that **simulation/test mode** is used, and a change window/approval path. `DLPEXO-ScopeHealth.ps1` reports the pilot group and the policy's current scope and mode. **Implement if missing:** Define the pilot group, deploy in simulation, and agree change control before enabling enforcement.

> ***Note on Exchange location scope.*** *In Purview DLP, the **Exchange** location is generally scoped as **All** (all mailboxes); a *pilot* is achieved by (a) running the policy in a **Test/simulation mode** and (b)*

*scoping the **rules** to the **pilot sender group** (a sender/`From member of` condition), then widening the rule scope as you expand. **Verify the exact scoping parameters in your tenant** — they evolve.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).** The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

Implementation Guide — Exchange Online DLP in Microsoft Purview

This is the ordered, do-it-in-this-sequence guide. It follows the standard model — **Assess** → **Design** → **Configure** → **Pilot (simulation/test)** → **Validate** → **Expand (enforce)** — with every object **named**, an explicit **pass/fail gate** at each phase, and a **rollback** for each change. Do **not** skip the simulation phase; DLP block/encrypt actions are high-impact.

Named objects used throughout (rename to your convention, but be consistent):

Object	Name used here
DLP policy (pilot)	`DLPEXO-Pilot`
DLP policy (production, post-pilot)	`DLPEXO-Exchange`
Sensitive → external, high volume rule	`DLPEXO-Rule-Ext-Block`
Sensitive → external, low volume rule	`DLPEXO-Rule-Ext-Notify`
Labeled mail → external rule	`DLPEXO-Rule-Label-Encrypt`
Sensitive → internal rule	`DLPEXO-Rule-Internal-Audit`
High-risk escalation rule	`DLPEXO-Rule-Ext-Escalate`
Pilot sender group (mail-enabled)	`SG-DLPEXO-Pilot`
Compliance notification mailbox	`dlp-alerts@contoso.com`
Trusted-partner domain (exception)	`partner.contoso.com`
Sensitivity label keyed on	`Confidential`
Example SITs keyed on	`Credit Card Number`, `U.S. Social Security Number (SSN)`

> ***Tooling note.*** *The **policy and rules** can be authored in the **Purview portal** *or* with* `New-/Set-DlpCompliancePolicy` *and* `New-/Set-DlpComplianceRule` *via* `Connect-IPPSSession`*. The provided* `DLPEXO-Configure.ps1` *builds them in **simulation** — it **previews with** `-WhatIf` **and writes nothing unless you pass** `-Commit`, and it **never enforces** (never sets* `Enable`/`Block`*). **Verify every portal path and cmdlet parameter against your tenant** — condition/action parameter names (e.g.* `-SentToScope`, `-NotifyAllowOverride`*, the encryption*

action) and the exact simulation mode values evolve.

Phase 0 — Assess (confirm you are ready)

1. **Confirm requirements.** Walk `02-requirements.md` R1–R8. Do **not** proceed until licensing

(incl. message encryption if you will encrypt), roles, in-scope sensitive content (SITs/labels), the recipient-scope decision, the per-scenario action matrix, the policy-tip plan, the transport-rule review, and a pilot sender group are all confirmed.

1. **Run Discovery.** `Connect-IPPSSession`, then `.\DLPEXO-Discovery.ps1 -IncludeSensitiveTypes` to

inventory existing DLP policies/rules scoped to **Exchange**, their **modes**, the SITs and labels available, and the transport rules that may interact. Keep the CSV as your baseline.

1. **Review transport-rule interaction (R7).** Reconcile any existing mail-flow rule that already

blocks/encrypts/redirects on content or recipient, so DLP and transport rules do not conflict.

Gate 0 (pass/fail): All of R1–R8 satisfied, Discovery baseline captured, transport-rule overlaps reconciled. **Fail** → **stop** and close the gap; do not configure.

Phase 1 — Design (name every object, agree every action)

1. **Confirm the content conditions (R3).** Fix the SITs (e.g. `Credit Card Number`,

`U.S. Social Security Number (SSN)`) with their **instance-count / confidence** thresholds, and the sensitivity label (`Confidential`) you will key on.

1. **Confirm the recipient scope + partner exception (R4).** External is the primary scope; list the

trusted-partner domains (`partner.contoso.com`) to add as **exceptions**.

1. **Confirm the per-scenario action matrix (R5):**

Rule	Condition	Enforce action (per decision)
`DLPEXO-Rule-Ext-Block`	SITs to **external**, **high** count (e.g. ≥ 10 instances), **except** partner domains	**Block** + policy tip + incident report
`DLPEXO-Rule-Ext-Notify`	SITs to **external**, **low** count (e.g. 1–9), **except** partner domains	**Policy tip / notify**, allow **override with justification**
`DLPEXO-Rule-Label-Encrypt`	`Confidential` label to **external**	**Encrypt** (Purview Message Encryption)
`DLPEXO-Rule-Internal-Audit`	SITs to **internal** recipients	**Notify / incident report** only (no block)
`DLPEXO-Rule-Ext-Escalate`	SITs to **external** at a **very high** count (e.g. ≥ 50)	**Block** + **escalate** incident report to `dlp-alerts@contoso.com`

1. **Confirm the policy-tip and notification plan (R6):** the tip text, override options, and the

notification / incident-report recipients (`dlp-alerts@contoso.com`).

1. **Confirm the pilot sender scope (R8):** `SG-DLPEXO-Pilot`.

Gate 1: Every object is named, every action is agreed **in writing**, the partner exception and pilot group exist. **Fail** → **do not configure**; close the design decision first.

Phase 2 — Configure the policy and rules in SIMULATION

Create **one** DLP policy, `DLPEXO-Pilot`, scoped to the **Exchange** location, in **simulation/test mode** (`TestWithNotifications` to also show policy tips, or `TestWithoutNotifications` for a silent pilot). Author in the portal, or run `.\DLPEXO-Configure.ps1 -WhatIf` to preview, then `.\DLPEXO-Configure.ps1 -Commit` (still simulation) to create the objects.

1. **Create the policy** (simulation, Exchange location):

• Location: **Exchange email**.

• Mode: `TestWithNotifications` (simulate *and* surface policy tips so the pilot senders see

them) — **verify the exact mode value in your tenant**.

1. **Add the five rules**, each with its **condition(s)** and the matching **action(s)**, keeping

enforcing actions OFF while in simulation (the block/encrypt take effect only when the policy Mode becomes `Enable` in Phase 4):

• `DLPEXO-Rule-Ext-Block` — condition: content **contains** the SITs at the **high** count

AND recipient **is external** (`-SentToScope NotInOrganization` — *verify*), **except** the trusted-partner domains (`-ExceptIfRecipientDomainIs partner.contoso.com` — *verify*). Configure the **block** action, **notify user (policy tip)**, and **generate incident report** — but these do not *act* until the policy is enabled.

• `DLPEXO-Rule-Ext-Notify` — same content/recipient at the **low** count; action: **notify

user (policy tip) **with** allow override / business justification** (`-NotifyAllowOverride WithJustification` — *verify*). No block.

• `DLPEXO-Rule-Label-Encrypt` — condition: message/attachment carries the `Confidential`

sensitivity label AND recipient is **external**; action: **encrypt** (the DLP encryption action — **verify the exact parameter/name for your tenant**, and confirm Purview Message Encryption is enabled, R1).

• `DLPEXO-Rule-Internal-Audit` — condition: SITs AND recipient **is internal**

(`-SentToScope InOrganization` — *verify*); action: **notify / generate incident report** only (visibility, no block).

• `DLPEXO-Rule-Ext-Escalate` — condition: SITs to external at a **very high** count; action:

block + incident report escalated to `dlp-alerts@contoso.com`.

1. **Scope the rules to the pilot senders.** Add a sender condition on `SG-DLPEXO-Pilot`

(`-FromMemberOf` — *verify*) so simulation only evaluates pilot senders' mail; widen this in Phase 5.

1. **Save.** The policy is now evaluating in simulation **without blocking or encrypting anyone.**

Gate 2: `DLPEXO-Pilot` exists, is in a **Test/simulation** mode, on the **Exchange** location, with all five rules present, each conditioned on sensitive content/recipient scope and scoped to the pilot senders. `DLPEXO-Validate.ps1` reports **PASS** for policy-exists, Exchange-location, mode-is-simulation, five-rules-present, and content-condition. **Fail** → **rollback:** `Set-DlpCompliancePolicy -Identity DLPEXO-Pilot -Mode Disable` (or `Remove-DlpCompliancePolicy`); simulation blocks nothing, so impact is limited to telemetry.

Phase 3 — Pilot & validate in simulation

1. Let the policy run in simulation on `SG-DLPEXO-Pilot` for the agreed observation window.
2. Perform the **sanctioned test actions** in `05-testing.md` from a pilot mailbox — send benign

test messages that match each scenario (a seeded SIT to an external test address at low and high counts, a `Confidential`-labeled message externally, a SIT to an internal recipient, and a message to a partner-domain address to confirm the exception).

1. Review the policy's **simulation results** and **Activity explorer**; run `.\DLPEXO-Report.ps1` to

roll up rule matches, policy-tip, and (in a Test-with-notifications pilot) tip events.

1. **Tune:** adjust SIT count/confidence thresholds and the partner-domain exception to remove false

positives, and confirm every intended scenario is **matched**.

Gate 3: Every one of the five scenarios registered its sanctioned test action in simulation (the rule *would* act), the **partner-domain exception** did **not** match, false positives triaged, and the **R5 action matrix** confirmed **in writing**. **Fail** → **do not enforce**; keep tuning in simulation. Rollback is unchanged from Phase 2 (disable/delete the policy).

Phase 4 — Enforce, then expand

Only after Gate 3 passes.

1. **Confirm each rule's enforce action** matches the **R5** matrix — **block** for

`DLPEXO-Rule-Ext-Block` / `-Ext-Escalate`, **encrypt** for `-Label-Encrypt`, **notify/override** for `-Ext-Notify`, **audit/notify** for `-Internal-Audit`.

1. **Move the policy out of simulation to enforce** (portal: turn the policy **On**; PowerShell:

`Set-DlpCompliancePolicy -Identity DLPEXO-Pilot -Mode Enable`). Do this **still scoped to the pilot senders** first.

1. **Re-run the sanctioned tests** (`05-testing.md`) and confirm the blocked/encrypted scenarios now

actually block/encrypt, the policy tip appears, override-with-justification works, and the partner-domain exception is still **not** blocked.

1. **Expand scope in rings.** Widen the rules' sender scope (add the next group, or remove the

`-FromMemberOf` pilot condition to cover **All** senders), re-validate, and repeat. Consider cloning/renaming to the production policy (`DLPEXO-Exchange`) as you exit pilot, per your naming standard.

Gate 4: On the pilot, each **block/encrypt** scenario enforces as designed, each **notify/audit** scenario records without blocking, override-with-justification is logged, and the partner exception holds; expansion rings each pass the same gate before widening. **Fail → rollback:** immediately return the policy to **simulation/test** (`Set-DlpCompliancePolicy -Mode TestWithNotifications`) or **Disable** it — this stops enforcement at once — then diagnose per `06-troubleshooting.md` before re-enforcing.

Rollback summary (fast reference)

Situation	Action
A rule blocks/encrypts legitimate mail	Set the policy to **simulation/test** (`Set-DlpCompliancePolicy -Mode TestWithNotifications`), then tune the SIT thresholds / add the recipient domain to the partner exception.
The whole policy misbehaves	`Set-DlpCompliancePolicy -Identity DLPEXO-Pilot -Mode Disable` (stops all enforcement) or `-Mode TestWithNotifications` (back to simulation).
A sender should not be in scope	Remove them from `SG-DLPEXO-Pilot`, or narrow the rule's `-FromMemberOf` condition.
A partner flow is being blocked	Add the recipient domain to the trusted-partner **exception** on the affected rule(s), re-validate in simulation.
Start over cleanly	`Remove-DlpComplianceRule` for each rule, then `Remove-DlpCompliancePolicy -Identity DLPEXO-Pilot`.

> *Enforcement changes are **high-impact** — make them under change control, keep the pilot ring small, and never move all scenarios to **Block/Encrypt** at once without validating each in simulation first. Where the encrypt action or a condition parameter name is uncertain, **build that rule in the portal first, then read it back with** `DLPEXO-Discovery.ps1` to learn the exact shape in your tenant.*

Testing & Validation — Exchange Online DLP

This is the pass/fail gate set. Run it in two passes: **once in simulation** (Phase 3 — the scenario is *matched* and the policy *would* act, nothing is blocked or encrypted) and **again after enforce** (Phase 4 — the block/encrypt scenarios *actually* act). Every test uses **benign test content** deliberately marked to match your policy conditions (e.g. a message carrying your `Confidential` sensitivity label and/or seeded with a matching sensitive information type such as a test credit-card pattern) — **never** real regulated data — sent to **test mailboxes** you control.

> ***Determination boundary.** These tests prove the **control works as configured**. Whether a given real-world message is a *violation*, and any consequence, is a **compliance / security / privacy / legal** determination — not a test outcome (see `global-requirements-purview.md` §5).*

Run `.\DLPEXO-Validate.ps1` for the configuration gates and `.\DLPEXO-Report.ps1` for the rule-match / policy-tip / block telemetry rollup between passes.

A. Configuration gates (automated — `DLPEXO-Validate.ps1`, read-only)

#	Check	PASS signal
A1	Policy exists	`DLPEXO-Pilot` is present.
A2	Exchange location	The policy covers the **Exchange** (email) location.
A3	Simulation first	In the pilot pass, the policy **Mode** is a **Test/simulation** mode (`TestWithNotifications` / `TestWithoutNotifications`), **not** `Enable`.
A4	Pilot scope	The rules are scoped to `SG-DLPEXO-Pilot` (sender condition), **not** all senders.
A5	Five rules present	`DLPEXO-Rule-Ext-Block`, `-Ext-Notify`, `-Label-Encrypt`, `-Internal-Audit`, `-Ext-Escalate` all exist and are enabled.
A6	Sensitive-content condition	Each content rule carries a sensitive-info-type and/or sensitivity-label condition (not "all mail").
A7	Partner exception referenced	The external rules reference the trusted-partner domain(s) as an **exception**, as designed.

Gate A: All applicable rows PASS. Any FAIL → fix in `03-implementation.md` before behavioural testing.

B. Behavioural tests — per scenario (manual, from a pilot mailbox)

For each scenario, send a test message and record the result. In the **simulation pass** the expected result is *"rule match recorded in simulation results / Activity explorer; message delivered, nothing blocked or encrypted"* (in `TestWithNotifications`, the **policy tip** may also appear). In the **enforce pass** the expected result is the **R5** action for that scenario.

#	Scenario	Test action (benign, matching test content)	Simulation PASS	Enforce PASS (per R5)
B1	**SIT → external, high count**	Send a message with **many** seeded SIT instances to an **external** test address	Match recorded; delivered	**Blocked**; policy tip + incident report generated
B2	**SIT → external, low count**	Send a message with **a few** seeded SIT instances externally	Match recorded; delivered	**Policy tip / warn**; **override with justification** works and is logged
B3	**Labeled → external**	Send a `Confidential`-labeled message externally	Match recorded; delivered	**Encrypted** (Purview Message Encryption)
B4	**SIT → internal**	Send seeded SIT content to an **internal** recipient	Match recorded; delivered	**Notify / incident report** only; **not** blocked
B5	**SIT → very high count → external**	Send a message exceeding the escalation threshold externally	Match recorded; delivered	**Blocked** + escalation incident report to `dlp-alerts@contoso.com`

Gate B (simulation pass): Every in-scope scenario **B1–B5** recorded the rule match. A scenario that records **nothing** is a coverage failure — do not enforce it; diagnose per `06-troubleshooting.md` (condition match, SIT threshold, recipient scope, latency).

Gate B (enforce pass): Every **block/encrypt** scenario enforces as designed **and** every **notify/audit** scenario records without blocking; **override with justification** is logged where configured. Any deviation → roll the policy back to simulation and tune.

C. Negative / false-positive tests (both passes)

#	Check	PASS signal
C1	Non-matching content	A message with **no** matching SIT/label triggers **no** DLP match.
C2	Trusted-partner exception	Sending matching content to a **trusted-partner** domain (`partner.contoso.com`) is **not** blocked/encrypted in the enforce pass.
C3	Out-of-scope sender	A sender **not** in `SG-DLPEXO-Pilot` shows **no** enforcement from this policy during the pilot.

#	Check	PASS signal
C4	Override / justification	Where **override with business justification** is configured (`DLPEXO-Rule-Ext-Notify`), the policy tip offers override and the override is **logged**.
C5	Transport-rule coexistence	An existing mail-flow rule and the DLP rule do **not** double-act or pre-empt each other unexpectedly (R7).

Gate C: All applicable rows PASS. A false positive (C1/C2 fails) must be tuned **before** wider enforcement — false positives erode user trust and drive shadow workarounds (e.g. senders switching to personal email).

D. Evidence to capture (for the defensibility pack)

Capture and retain, for each pass:

- `DLPEXO-Validate.ps1` output (configuration gate results).
- `DLPEXO-Report.ps1` output (policy modes + rule-match / policy-tip / block export, where available).
- Screenshots of **Purview** → **Activity explorer / DLP simulation results** showing each scenario's recorded match, and of the **policy tip / block / encryption** behavior in the enforce pass.
- A completed **B-scenario matrix** signed off by the change owner, plus the **R5 action decision**, the **R4 recipient-scope + partner-exception decision**, and the **R7 transport-rule reconciliation**.

> *This evidence set is what makes the control **defensible** — it shows *what* was configured, *that* each scenario was validated in simulation before enforcement, and *that* enforcement behaves as designed. Store it with the change record.*

Troubleshooting — Exchange Online DLP

The failures you will actually hit, with the most likely cause and the fix. Work top-down: **most "the policy isn't working" reports trace back to condition matching, recipient scope, simulation-vs- enforce confusion, or a conflicting transport rule — not the rule itself.** Where a portal path or cmdlet parameter is named, **verify it against your tenant** — the Purview portal and DLP condition/action options evolve.

1. A test message doesn't match at all

Symptoms: Send a message you expect to match and **nothing** appears in the DLP simulation results / Activity explorer; the policy never fires.

Likely causes & fixes

• **Content doesn't carry the SIT/label.** The test message must actually contain the **sensitive information type **(a valid pattern that meets the SIT's checksum/format) or carry the** sensitivity label** the rule keys on. Seed a known-good match and retry. `DLPEXO-Discovery.ps1 -IncludeSensitiveTypes` lists the SITs available.

• **Instance count below threshold.** A rule that requires ≥ N instances will not fire on fewer. Check the count you set (`DLPEXO-Rule-Ext-Block` high, `-Ext-Notify` low) versus the test content.

• **Recipient scope mismatch.** An "external" rule (`-SentToScope NotInOrganization`) will not match a message sent **internally** (and vice-versa). Confirm you sent to the right recipient type.

• **Sender out of pilot scope.** During the pilot the rules are scoped to `SG-DLPEXO-Pilot` (`-FromMemberOf`). A sender not in that group is not evaluated. `DLPEXO-ScopeHealth.ps1` shows each rule's sender scope; `DLPEXO-Validate.ps1` gate **A4** confirms it.

• **Propagation delay.** New/changed policies take time to reach the transport pipeline. Wait, then retest.

2. Policy tips don't appear in Outlook

Symptoms: The rule matches (visible in simulation results) but the sender never sees a **policy tip**.

Likely causes & fixes

• **Policy in a silent test mode.** `TestWithoutNotifications` simulates **without** showing tips. Use `TestWithNotifications` if you want tips during the pilot. `DLPEXO-Validate.ps1` gate **A3** shows the mode.

• **NotifyUser not configured.** The rule must notify the sender for a tip to show. Confirm the rule's user-notification / policy-tip action is set.

• **Client support varies.** Policy-tip rendering differs across Outlook desktop / web / mobile. **Verify tip behavior per client in your tenant** — a tip missing on one client is not proof the rule is broken.

• **Latency.** Tip configuration changes take time to propagate to clients. Wait and retest.

3. The rule fires on the wrong things (false positives)

Symptoms: Legitimate business mail is flagged/blocked — a sanctioned partner flow, an internal message, a low-risk email.

Likely causes & fixes

• **Missing/stale trusted-partner exception.** Add the recipient domain to the exception (`-ExceptIfRecipientDomainIs`) on the external rules and confirm it is referenced. `DLPEXO-Validate.ps1` gate **A7** checks the exception is present.

• **SIT count/confidence too loose.** Raise the **instance count** or **confidence level** so ordinary

mail with an incidental match doesn't trip the rule, then re-test.

• **Condition too broad.** A rule keyed on "all mail" instead of a **SIT/label** matches everything.

Tighten the condition (R3). `DLPEXO-Validate.ps1` gate **A6** flags rules with no readable content condition.

• **Tune in simulation** — never tune by loosening an already-enforcing block in production.

4. Simulation looks fine but enforcement doesn't block/encrypt

Symptoms: In simulation the match is recorded; after "turning it on" the block/encrypt still doesn't happen.

Likely causes & fixes

• **Policy still in a Test mode.** Simulation records but does **not** block or encrypt. To enforce,

move the policy out of simulation: portal → turn the policy **On**; PowerShell → `Set-DlpCompliancePolicy -Identity DLPEXO-Pilot -Mode Enable`. `DLPEXO-Validate.ps1` gate **A3** shows the current mode.

• **Enforcing action not set on the rule.** Simulation configures conditions; the **Block** /

Encrypt action must be present on the rule for enforcement to act. Confirm the rule's action matches the R5 matrix (`DLPEXO-Configure.ps1` deliberately leaves enforcing actions off — you set them in Phase 4).

• **Sender/recipient out of scope.** Enforcement only applies within the rule's sender/recipient scope.

Confirm with `DLPEXO-ScopeHealth.ps1` / gate **A4**.

• **Propagation delay.** Mode/action changes take time to reach transport. Wait, then retest.

5. The encrypt action doesn't apply (or errors)

Symptoms: The labeled-mail-to-external rule (`DLPEXO-Rule-Label-Encrypt`) matches but the message is not encrypted, or configuring the encrypt action errors.

Likely causes & fixes

• **Message encryption not enabled.** The DLP encrypt action depends on **Purview Message Encryption /

Azure Rights Management** being enabled in the tenant (R1). `DLPEXO-ScopeHealth.ps1` flags IRM/OME readiness; if not enabled, enable it before relying on the encrypt action.

• **Wrong action parameter/name.** The DLP encryption action parameter/name **evolves** and differs

across tenants. **Build the encrypt rule in the portal first**, then read it back with `DLPEXO-Discovery.ps1` to learn the exact shape, rather than guessing the parameter.

• **A transport rule is also stamping encryption.** Two controls encrypting the same mail causes

confusing double-actions — reconcile per §7 / R7.

6. `New-DlpComplianceRule` fails on a condition/action parameter

Symptoms: `DLPEXO-Configure.ps1 -Commit` (or manual scripting) errors on a parameter such as `-SentToScope`, `-ExceptIfRecipientDomainIs`, `-FromMemberOf`, or the sensitive-information condition.

Fix

- **Verify the exact parameter names/values for your tenant.** These parameter names and permitted values **evolve**. Run `Get-Help New-DlpComplianceRule -Full` and check current Microsoft docs, then adjust. `DLPEXO-Configure.ps1` deliberately **prints the plan** and wraps each write in try/catch so a differing parameter **warns** rather than aborting the whole run.
- **Prefer the portal for the first build** if PowerShell parameter names are uncertain, then read the result back with `DLPEXO-Discovery.ps1` to learn the exact shape in your tenant.

7. DLP and an Exchange transport (mail-flow) rule conflict

Symptoms: A message is double-acted on (e.g. encrypted twice, or blocked with an unexpected message), or DLP "never fires" because a transport rule already deleted/redirected the mail first.

Likely causes & fixes

- **Overlapping controls.** DLP and transport rules **coexist** and both act in transport. Review transport rules that block/encrypt/delete/redirect on content or recipient (`Get-TransportRule`, or `DLPEXO-Discovery.ps1` / `DLPEXO-ScopeHealth.ps1` flag them) and **decide which control owns each behavior** (R7). Remove or adjust the duplicate.
- **Pre-emption.** A transport rule that **deletes or redirects** a message *before* DLP evaluates it makes DLP look broken. Reorder or narrow the transport rule so DLP can evaluate.

8. Multiple DLP policies overlap / precedence confusion

Symptoms: A rule you expect to act doesn't, or a different policy's action takes effect.

Fix

- **Check policy precedence.** When several DLP policies cover Exchange, **priority order** determines which acts. `DLPEXO-ScopeHealth.ps1` lists Exchange-location policies by **Priority** — review for overlap/shadowing and adjust priority so the intended policy/rule wins.

9. Activity report is empty or export fails

Symptoms: `DLPEXO-Report.ps1` returns no activity, or the Activity Explorer export warns and skips.

Likely causes & fixes

• **Licensing/permission.** `Export-ActivityExplorerData` is licensing-dependent; the script **warns

and degrades to policy-modes only **rather than failing. Fall back to** Purview → Data loss prevention (Policy / Activity explorer)** in the portal.

• **Quiet pilot window.** A small pilot may simply have no activity in the window — widen `-Days` or

perform the sanctioned test actions from `05-testing.md` first.

• **Role.** Ensure the connected account holds a role that can read Activity Explorer / DLP reports.

10. "It blocked an exec / broke a business flow" — emergency rollback

Do this immediately, then diagnose:

• Return the policy to **simulation** or **off**:

`Set-DlpCompliancePolicy -Identity DLPEXO-Pilot -Mode TestWithNotifications` (simulation) or `-Mode Disable` (off). Either **stops enforcement at once.**

• Or add the affected recipient domain to the **trusted-partner exception** on the offending rule.

• Then work the relevant section above (usually §3 false positives, §7 transport conflict, or §4

scope), fix **in simulation**, re-validate with `DLPEXO-Validate.ps1`, and only then re-enforce.

When to escalate to the determination owners (not a technical fix)

Some questions are **not** troubleshooting — they are **determinations** that belong to compliance / security / privacy / legal (see `global-requirements-purview.md` §5):

• *Should* this scenario be **blocked/encrypted** or only **audited**? → the **R5** decision owners.

• Is *this content* actually sensitive / in scope? → **compliance / data owners**.

• Is a recipient **domain trusted**? → **compliance / security** (R4).

• Is a user's message a **violation**, and what is the consequence? → **HR / legal**.

Provide them the **evidence** (`DLPEXO-Report.ps1`, Activity explorer, the validation output) — not a verdict.

Best Practices — Exchange Online DLP

Field-tested guidance for running Exchange Online DLP well. It complements — does not replace — the implementation steps and the pass/fail gates.

Deploy the way the house model says — simulate, then enforce

• **Never enforce a scenario you have not validated in simulation.** The single most common cause of a

DLP incident is a **Block** or **Encrypt** rule moved to enforce without a simulation pass. Every scenario goes **simulation (Test)** → **validate** → **enforce**, one at a time.

- **Small pilot ring first.** Scope the rules to a handful of pilot senders (`SG-DLPEXO-Pilot`) before

the whole tenant. It is far cheaper to find a false positive on ten senders than ten thousand.

- **Enforce one scenario at a time.** Do not flip external-block, encrypt, and escalate to enforce

together. Enforce the highest-risk scenario first (often SIT→external block), validate, then the next.

- **Keep a rollback ready.** `Set-DlpCompliancePolicy -Mode TestWithNotifications` (back to simulation)

or `-Mode Disable` stops enforcement instantly — know the command before you enforce.

Key the rules on *sensitive* content and the right recipient scope

- **Condition on SITs and/or sensitivity labels**, not "all mail." A policy that blocks *any* external

message generates noise and workarounds; one that acts on **classified/labeled** content is defensible and proportionate.

- **Tune SIT confidence and instance count** deliberately. A low count is usually a **warn/notify**

(policy tip, allow override); a high count is a **block**. Set the thresholds with the data owners, not by guesswork, and raise confidence to cut false positives before you enforce.

- **Lead with sensitivity labels where they exist** — a label is an explicit business decision about

sensitivity and travels with the message/attachment. Fall back to SITs where labels are not yet adopted, and add label conditions as labeling matures.

- **Scope by recipient.** External egress is the primary risk; treat internal as **visibility/audit**

unless policy says otherwise.

Maintain the trusted-partner exception deliberately

- **The partner-domain exception is what lets you block the bad path without blocking the business.**

Keep the list current (`partner.contoso.com` and peers) and **review it on a schedule** — a stale or missing exception is both a false-positive source and, if over-broad, a bypass.

- **Whether a domain is "trusted" is a business/compliance decision** — capture it in writing (R4),

don't add domains ad hoc to silence a ticket.

Use policy tips as user education, not just enforcement

- **Write clear policy-tip text** that tells the sender *what* matched and *what to do* (remove the

data, encrypt, or override with justification). A good tip prevents the incident and the support ticket.

- **Prefer warn-with-override for low-severity scenarios** so a legitimate business need is not hard

blocked — the override is logged as evidence and surfaces genuine needs that may belong in an exception.

- **Confirm tip rendering per Outlook client** (desktop / web / mobile) — support varies; do not assume every client shows the tip identically.

Reconcile with Exchange mail-flow (transport) rules

- **DLP and transport rules coexist and both act in transport.** Before enforcing, review transport rules (`Get-TransportRule`) for any that already block/encrypt/redirect on content or recipient, and **decide which control owns each behavior** — two controls stamping encryption or one deleting mail before DLP evaluates causes confusing double-actions or silent pre-emption.
- **Prefer Purview DLP as the strategic home** for content-based data-loss control across the estate; keep transport rules for mail-routing/hygiene concerns DLP does not cover.

Operate it, don't set-and-forget

- **Review DLP activity regularly** and roll up with `DLPEXO-Report.ps1`. Rising matches on a scenario are a signal to review scope, tuning, or user education — not automatically a reason to block harder.
- **Watch override / justification trends.** A scenario that is constantly overridden may be mis-scoped or may indicate a genuine business flow that belongs in the partner exception, not a block.
- **Version and name policies clearly.** Move from `DLPEXO-Pilot` to a production `DLPEXO-Exchange` as you exit pilot; keep a change log of scope, threshold, and action changes.
- **Re-validate after portal changes.** The Purview portal and DLP condition/action options evolve; re-run `DLPEXO-Validate.ps1` after Microsoft feature updates and confirm nothing regressed.

Keep the roles and determinations where they belong

- **Least privilege, split duties.** DLP authoring and Exchange mail-flow administration are different roles — do not fold them into one over-privileged account.
- **Determinations stay with the owners.** *Which* data is sensitive, *which* scenarios to block or encrypt, *whether* a domain is trusted, and *whether* a message is a violation are **compliance / security / privacy / legal** calls. You provide the configuration and the evidence, not the judgement (see `global-requirements-purview.md` §5).
- **Communicate before you enforce.** Policy tips and user notice reduce friction and support the lawful, transparent operation of a data-loss control; surprise blocks generate tickets and workarounds.

Security & handling of the tooling

- **Read-only / `-WhatIf` by default.** Run the provided scripts in their safe default first; only add

`-Commit` when you intend to write, under change control.

• **No secrets in code.** The scripts use interactive modern-auth (`Connect-IPPSSession`); never hardcode credentials, tokens, or app secrets.

• **Least-privilege connections.** Connect with the least-privilege role that the task needs, not a standing Global Administrator session.

Deliverables Summary — Exchange Online DLP in Microsoft Purview

Everything included in this solution, what each piece is for, and how it maps to the deployment model. Use it as the table of contents and the "did we finish?" checklist.

Documents

Document	What it gives you
01 — Solution Overview	What Exchange Online DLP delivers, the five email scenarios (SIT→external high/low, labeled→external, SIT→internal, partner exception), policy tips, transport-rule interaction, the simulate-first rule, and the Assess→Design→Configure→Pilot→Validate→Expand model.
02 — Requirements	The solution-specific prerequisites (R1–R8): DLP-capable licensing (+ message encryption), DLP/mail-flow roles, in-scope SITs/labels and thresholds, the recipient-scope + partner-domain decision, the per-scenario action decision, the policy-tip/notification plan, transport-rule reconciliation, and a pilot sender group — each with Verify / Implement-if-missing. References the category `global-requirements-purview.md`.
03 — Implementation Guide	The ordered build: assess → design (name every object + action matrix) → author the policy + five rules **in simulation** → pilot & validate → enforce & expand. Every object named, with a pass/fail gate and rollback per phase.
05 — Testing & Validation	The pass/fail gates: configuration gates (A), per-scenario behavioural tests (B) run in simulation then enforce, false-positive/negative tests (C) incl. the partner exception and transport-rule coexistence, and the evidence to capture (D).

Document	What it gives you
06 — Troubleshooting	The failures you will actually hit — no match, tips not showing, false positives, enforce-not-blocking, encryption not applying, transport-rule conflict, empty report — each with cause and fix, referencing the five scripts by name.
07 — Best Practices	How to run it well: simulate-then-enforce discipline, keying on sensitive content + recipient scope, maintaining the partner exception, policy tips as education, transport-rule reconciliation, ongoing operation, and role/determination boundaries.
Executive Summary	The one-page business-level summary for stakeholders and sign-off.

PowerShell tools (5)

All are **read-only or** `-WhatIf` **by default**, use interactive modern-auth (`Connect-IPPSSession`), and contain **no credentials**. They use standard Security & Compliance PowerShell cmdlets and do **not** invent cmdlets.

Script	Purpose	Safe default
DLPEXO-Discovery.ps1	Inventory existing DLP policies/rules scoped to **Exchange** (with mode), the transport-rule interaction, and the sensitivity labels + sensitive info types available to key on — the baseline of what already protects email.	Read-only
DLPEXO-Configure.ps1	Build/update the `DLPEXO-Pilot` policy and the five scenario rules **in simulation**; previews with `-WhatIf` and writes nothing unless you pass `-Commit`. Never enforces.	`-WhatIf` (preview)
DLPEXO-Validate.ps1	Run the configuration gates (policy exists, Exchange location, simulation-first, pilot scope, five rules, sensitive-content conditions, partner exception) with PASS/FAIL output.	Read-only

Script	Purpose	Safe default
DLPEXO-Report.ps1	Roll up policy modes and, where available, **DLP rule-match / policy-tip / block events** via Activity-explorer export — the operational + evidence view.	Read-only
DLPEXO-ScopeHealth.ps1	Report Exchange DLP scope health — mail-flow coverage, policy precedence/overlap with other DLP + transport rules, and licensing-readiness flags — and preview any scope change with `-WhatIf`.	`-WhatIf` / read-only

How it maps to the deployment model

Phase	Deliverables used
Assess	01, 02, `DLPEXO-Discovery.ps1`, `DLPEXO-ScopeHealth.ps1`
Design	02, 03 (named objects + action matrix), 07
Configure	03, `DLPEXO-Configure.ps1 -WhatIf` then `-Commit` (simulation)
Pilot (simulation)	03, 05 (Gate B simulation pass), `DLPEXO-Report.ps1`
Validate	05, `DLPEXO-Validate.ps1`, `DLPEXO-Report.ps1`
Expand (enforce)	03 (Phase 4), 05 (Gate B enforce pass), 06, 07

Definition of done

• Transport-rule interaction **reconciled** (R7) and in-scope SITs/labels + thresholds confirmed.

• `DLPEXO-Pilot` on the **Exchange** location with five rules built **in simulation**, scoped to the pilot senders, all configuration gates (A) PASS.

• Behavioural gates (B) PASS in the **simulation pass**, then in the **enforce pass** for the scenarios the R5 decision blocks/encrypts; false-positive gates (C) — including the **partner exception** — PASS.

• **Evidence pack** (D) captured and stored with the change record.

• Regulatory / policy determinations (which data, which action, which domains are trusted) recorded by the **compliance / security** owners — not by the administrator.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPEXO-Configure.ps1

Purpose: Creates (or updates) an Exchange Online DLP compliance policy scoped to the Exchange (email) location, in SIMULATION/TEST mode, plus five rules — sensitive-info-to-external (high & low volume), labeled-mail- to-external (encrypt), sensitive-info-to-internal (audit), and a high-count escalation — each conditioned on sensitive content and recipient scope, with policy tips. SAFE BY DEFAULT: previews with -WhatIf and writes NOTHING unless you pass -Commit. It never enforces (never sets Enable/BlockAccess); moving to enforce is a deliberate portal/PowerShell step in Phase 4 of the implementation guide. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPEXO-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPEXO-Configure.ps1 -Commit # create the policy + rules in SIMULATION .\DLPEXO-Configure.ps1 -Commit -PilotGroup "SG-DLPEXO-Pilot" -LabelName "Confidential" -PartnerDomain "partner.contoso.com" EXPECTED RESULTS With -Commit: policy 'DLPEXO-Pilot' exists in a Test/simulation mode, Exchange location, with five rules scoped to the pilot senders and enforcing actions OFF. Validate with DLPEXO-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The exact condition/action parameter names (e.g. -SentToScope, -NotifyAllowOverride, the encryption action) and the simulation Mode value EVOLVE — the script uses documented names, PRINTS the plan for you to verify with `Get-Help New-DlpComplianceRule -Full` against your tenant, and wraps each write in try/catch so a parameter that differs in your tenant WARNS rather than aborting. It does NOT enforce (no Enable / BlockAccess=$true). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "DLPEXO-Pilot", [string]$PilotGroup = "SG-DLPEXO-Pilot", # pilot sender group (rule-level -FromMemberOf scope) [string]$LabelName = "Confidential", # sensitivity label the encrypt rule keys on [string]$PartnerDomain = "partner.contoso.com", # trusted-partner exception domain [string]$NotifyMailbox = "dlp-alerts@contoso.com", # compliance mailbox for incident reports [string[]]$SitNames = @("Credit Card Number","U.S. Social Security Number (SSN)"), [int]$LowCount = 1, # low-volume threshold (warn/notify) [int]$HighCount = 10, # high-volume threshold (block) [int]$EscalateCount = 50, # very-high threshold (block + escalate) [string]$SimMode = "TestWithNotifications" # simulation + policy tips; verify exact value in your tenant)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPEXO-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPEXO-Configure.ps1 -Commit # create the policy + rules in SIMULATION .\DLPEXO-Configure.ps1 -Commit -PilotGroup "SG-DLPEXO-Pilot" -LabelName "Confidential" -PartnerDomain "partner.contoso.com" EXPECTED RESULTS With -Commit: policy 'DLPEXO-Pilot' exists in a Test/simulation mode, Exchange location, with five rules scoped to the pilot senders and enforcing actions OFF. Validate with DLPEXO-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The exact condition/action parameter names (e.g. -SentToScope, -NotifyAllowOverride, the encryption action) and the simulation Mode value EVOLVE — the script uses documented names, PRINTS the plan for you to verify with `Get-Help`

`New-DlpComplianceRule -Full` against your tenant, and wraps each write in try/catch so a parameter that differs in your tenant WARNS rather than aborting. It does NOT enforce (no Enable / BlockAccess=$true). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "DLPEXO-Pilot", [string]$PilotGroup = "SG-DLPEXO-Pilot", # pilot sender group (rule-level -FromMemberOf scope) [string]$LabelName = "Confidential", # sensitivity label the encrypt rule keys on [string]$PartnerDomain = "partner.contoso.com", # trusted-partner exception domain [string]$NotifyMailbox = "dlp-alerts@contoso.com", # compliance mailbox for incident reports [string[]]$SitNames = @("Credit Card Number","U.S. Social Security Number (SSN)"), [int]$LowCount = 1, # low-volume threshold (warn/notify) [int]$HighCount = 10, # high-volume threshold (block) [int]$EscalateCount = 50, # very-high threshold (block + escalate) [string]$SimMode = "TestWithNotifications" # simulation + policy tips; verify exact value in your tenant)

DLPEXO-Discovery.ps1

Purpose: Read-only baseline discovery for Exchange Online DLP. Inventories the DLP compliance policies and rules already in the tenant, flags which cover the Exchange (email) location and their Mode (simulation/test vs enable), surfaces Exchange mail-flow (transport) rules that already block / encrypt / redirect (so DLP does not conflict with them), and lists the sensitivity labels and sensitive information types the rules could key on. This is the "what already protects email" baseline before you build anything. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). For transport-rule signals also connect Exchange Online (Connect-ExchangeOnline): Connect-IPPSSession .\DLPEXO-Discovery.ps1 .\DLPEXO-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPEXO-Discovery.csv) of DLP policies (with an ExchangeLocation flag + Mode), DLP rules, labels, and any content/encryption transport rules; with -IncludeSensitiveTypes, sensitive info types to .\DLPEXO-SensitiveTypes.csv. A tenant with NO Exchange-location DLP policy is itself the first finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell (Get-DlpCompliancePolicy/-Rule, Get-Label, Get-DlpSensitiveInformationType) and, if available, Exchange Online (Get-TransportRule); none are invented. DLP is licensing-dependent — the script WARNS where a feature is not readable rather than assuming. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPEXO-Discovery.csv", [string]$SensitiveOutPath = ".\DLPEXO-SensitiveTypes.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). For transport-rule signals also connect Exchange Online (Connect-ExchangeOnline): Connect-IPPSSession .\DLPEXO-Discovery.ps1 .\DLPEXO-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPEXO-Discovery.csv) of DLP policies (with an ExchangeLocation flag + Mode), DLP rules, labels, and any content/encryption transport rules; with -IncludeSensitiveTypes, sensitive info types to .\DLPEXO-SensitiveTypes.csv. A tenant with NO Exchange-location DLP policy is itself the first finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell (Get-DlpCompliancePolicy/-Rule, Get-Label, Get-DlpSensitiveInformationType) and, if available, Exchange Online (Get-TransportRule); none are invented. DLP is licensing-dependent — the script WARNS where a feature is not readable rather than assuming. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPEXO-Discovery.csv", [string]$SensitiveOutPath = ".\DLPEXO-SensitiveTypes.csv")

DLPEXO-Report.ps1

Purpose: Read-only operational + evidence report for Exchange Online DLP. Summarizes the DLP policies that cover the Exchange location and their mode (simulation vs enforce), and — where the tenant/licensing exposes it — exports recent DLP activity from Activity Explorer (rule matches, policy-tip shown, override, and block events) so you can see what the policy is recording. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPEXO-Report.ps1 .\DLPEXO-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\DLPEXO-PolicyModes.csv (policies + mode + Exchange flag) and, where available, .\DLPEXO-Activity.csv (Activity Explorer DLP events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns) rather than assuming availability. For DLP reports the Purview portal (Data loss prevention > Policy / Activity explorer) is the authoritative view. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPEXO-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPEXO-Activity.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPEXO-Report.ps1 .\DLPEXO-Report.ps1 -Days 7 EXPECTED RESULTS A console summary + .\DLPEXO-PolicyModes.csv (policies + mode + Exchange flag) and, where available, .\DLPEXO-Activity.csv (Activity Explorer DLP events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent — the script degrades gracefully (warns) rather than assuming availability. For DLP reports the Purview portal (Data loss prevention > Policy / Activity explorer) is the authoritative view. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPEXO-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPEXO-Activity.csv")

DLPEXO-ScopeHealth.ps1

Purpose: Scope and health helper for Exchange Online DLP. Reports which DLP policies cover the Exchange location and their PRECEDENCE/priority (so you can see overlap/ordering), flags transport (mail-flow) rules that may conflict with DLP, checks message-encryption readiness for the encrypt scenario, and shows the pilot policy's current mode + rule sender-scope — so you can see what the policy WOULD cover before you widen it. Read-only by default; any scope change is preview-only via -WhatIf (this tool does not itself widen scope — it prints the command for you to run under change control). HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). For transport-rule / encryption signals also connect Exchange Online (Connect-ExchangeOnline): Connect-IPPSSession .\DLPEXO-ScopeHealth.ps1 .\DLPEXO-ScopeHealth.ps1 -PolicyName "DLPEXO-Pilot" -PilotGroup "SG-DLPEXO-Pilot" -WhatIf EXPECTED RESULTS A console summary: Exchange-location DLP policies with priority (precedence), overlapping transport rules, message-encryption readiness (IRM/OME), and the pilot policy's mode + rule scope. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Get-DlpCompliancePolicy / Get-DlpComplianceRule and, if available, Get-TransportRule / Get-IRMConfiguration; none invented. Precedence/priority and encryption readiness are signals — the authoritative view is the Purview / Exchange admin portal. > [CmdletBinding()] param([switch]$WhatIf, [string]$PolicyName = "DLPEXO-Pilot", [string]$PilotGroup = "SG-DLPEXO-Pilot")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession). For transport-rule / encryption signals also connect Exchange Online (Connect-ExchangeOnline): Connect-IPPSSession .\DLPEXO-ScopeHealth.ps1 .\DLPEXO-ScopeHealth.ps1 -PolicyName "DLPEXO-Pilot" -PilotGroup "SG-DLPEXO-Pilot" -WhatIf EXPECTED RESULTS A console summary: Exchange-location DLP policies with priority (precedence), overlapping transport rules, message-encryption readiness (IRM/OME), and the pilot policy's mode + rule scope. With -WhatIf, a PREVIEW of the scope-widening command (nothing is changed). .NOTES White-label Microsoft Purview tooling. Read-only / -WhatIf only. No credentials stored. Uses standard Get-DlpCompliancePolicy / Get-DlpComplianceRule and, if available, Get-TransportRule / Get-IRMConfiguration; none invented. Precedence/priority and encryption readiness are signals — the authoritative view is the Purview / Exchange admin portal. > [CmdletBinding()] param([switch]$WhatIf, [string]$PolicyName = "DLPEXO-Pilot", [string]$PilotGroup = "SG-DLPEXO-Pilot")

DLPEXO-Validate.ps1

Purpose: Read-only validation of the Exchange Online DLP policy against the configuration gates in 05-testing.md (Gate A). Confirms the policy exists, covers the Exchange (email) location, is in a Test/simulation mode (during the pilot pass), is scoped to a pilot sender group (not all senders), has the five expected scenario rules present and enabled, each carries a sensitive-content condition, and the external rules reference the trusted-partner exception. Prints PASS/FAIL per check. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPEXO-Validate.ps1 .\DLPEXO-Validate.ps1 -PolicyName "DLPEXO-Pilot" -PartnerDomain "partner.contoso.com" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A3 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* cmdlets; none invented. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. Some conditions (labels, exceptions) surface under different properties across tenants — where blank, the script flags them for a manual portal check rather than failing. > [CmdletBinding()] param([string]$PolicyName = "DLPEXO-Pilot", [string]$PartnerDomain = "partner.contoso.com", [string]$PilotGroup = "SG-DLPEXO-Pilot", [string[]]$ExpectedRules = @("DLPEXO-Rule-Ext-Block","DLPEXO-Rule-Ext-Notify","DLPEXO-Rule-Label-Encrypt","DLPEXO-Rule-Internal-Audit","DLPEXO-Rule-Ext-Escalate"))

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPEXO-Validate.ps1 .\DLPEXO-Validate.ps1 -PolicyName "DLPEXO-Pilot" -PartnerDomain "partner.contoso.com" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A3 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* cmdlets; none invented. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. Some conditions (labels, exceptions) surface under different properties across tenants — where blank, the script flags them for a manual portal check rather than failing. > [CmdletBinding()] param([string]$PolicyName = "DLPEXO-Pilot", [string]$PartnerDomain = "partner.contoso.com", [string]$PilotGroup = "SG-DLPEXO-Pilot", [string[]]$ExpectedRules = @("DLPEXO-Rule-Ext-Block","DLPEXO-Rule-Ext-Notify","DLPEXO-Rule-Label-Encrypt","DLPEXO-Rule-Internal-Audit","DLPEXO-Rule-Ext-Escalate"))